

Machine Learning

Gradient Descent in Practice



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

- How much data goes into one gradient step: batch, stochastic, mini-batch.
- What an *epoch* is, and how many updates it contains.
- Why gradient descent needs feature scaling, shown geometrically.

Data Per Step

The gradient descent

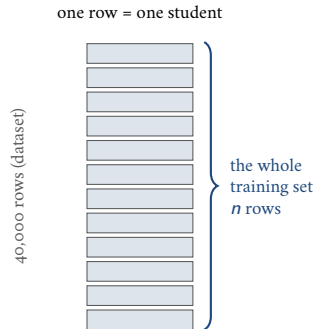
$$\theta^{(i+1)} = \theta^{(i)} - \eta \nabla_{\theta} J(\theta^{(i)}), \quad \nabla_{\theta} J = \frac{2}{n} \mathbf{X}^{\top} (\mathbf{X}\theta - \mathbf{y})$$

- Observation: n in the gradient denotes how many rows are used to compute one step.
- Traditional Optimization: Use all of them.

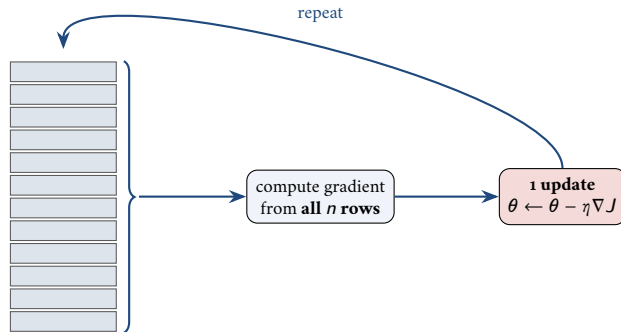
Variants

How many rows you put into it changes.
That single choice gives three different algorithms.

The training set, drawn as a stack of rows



Method 1 (Traditional) — Batch gradient descent



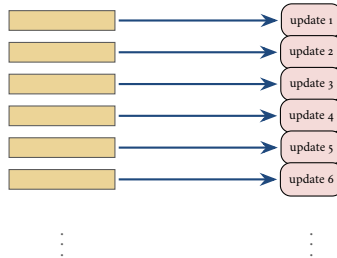
- All 40,000 rows contribute to **every** step.
- Each step costs a full pass over the data.

Count

1 epoch = 1 update.
1000 epochs = **1000** updates.

Method 2 — Stochastic gradient descent (SGD)

one row at a time



n updates
in a single epoch

- Each observation causes an update.
- Enormously many updates, each almost free.
- But if one observation is bad $\implies$ *terrible* estimate of the whole gradient

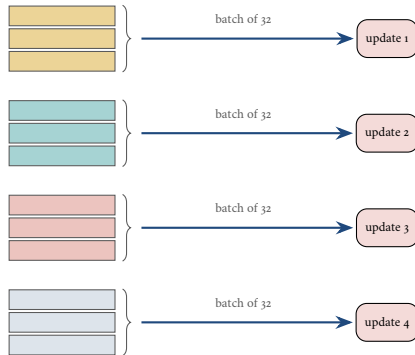
Observation

1 epoch = **40,000** updates.

It may never settle exactly at the minimum, only bounce around it.

Method 3 — Mini-batch gradient descent

rows split into equal chunks



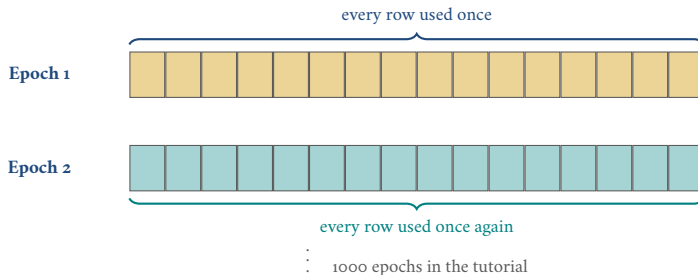
$n/32$ updates
in a single epoch

Count

$40,000 \div 32 = 1,250$ updates in one epoch. Over 1000 epochs that is **1,250,000** updates.

What Exactly Is an Epoch?

One epoch = the model has seen every row exactly once



The word people get wrong

An epoch is **not** an update. It is *one complete pass over the data*.

How many updates happen inside that pass depends entirely on the batch size.

Method	Updates per epoch
Batch	1
Mini-batch (32)	1,250
SGD	40,000

Updates in Epoch

$$\text{updates per epoch} = \left\lceil \frac{n}{\text{batch size}} \right\rceil$$

$$\text{total updates} = \text{epochs} \times \left\lceil \frac{n}{\text{batch size}} \right\rceil$$

With $n = 40,000$, batch size 32, and 1000 epochs:

$$1000 \times \frac{40,000}{32} = 1,250,000$$

Batch size	Updates/epoch	Rows per update
1	40,000	1
32	1,250	32
256	157	256
4,096	10	4,096
40,000	1	40,000

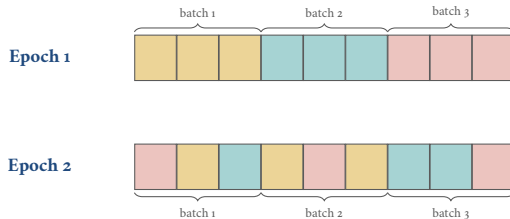
Observation

SGD: Batch size = 1.

GD: Batch size = n .

Everything between is mini-batch.

Why we reshuffle at the start of every epoch



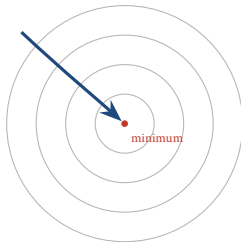
same rows,
different groups

- Without reshuffling, the same 32 observations would travel together in every single epoch.
- The model would then keep seeing the same 1,250 gradients over and over, and could learn the quirks of those particular groupings.
- If the file happened to be sorted — Branchwise — it would be far worse: early batches would be all one stream.
- Fix: `rng.permutation(n)` **inside** the epoch loop.

The Three Paths

The same hill, three different walks

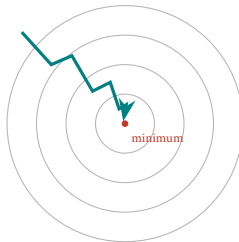
Batch



Batch

Straight, smooth, few steps. Each step is expensive but correct.

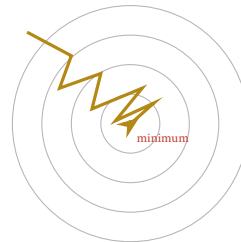
Mini-batch



Mini-batch

Slight wobble, many cheap steps. Arrives quickly.
Used in practice.

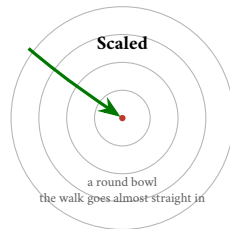
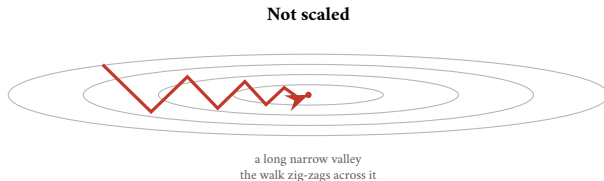
SGD



SGD

Very noisy. Gets near fast, then rattles around the minimum.

Failure 1 — Features on different scales



- `AptitudeScore` runs to 100; `SoftSkillsRating` runs to 5. The cost surface becomes a long narrow valley.
- Gradient descent uses one η for every direction at once. A step safe along the narrow axis is far too small along the long one, and vice versa.
- `StandardScaler` makes every axis comparable, turning the valley into a bowl.

Failure 2 — The learning rate

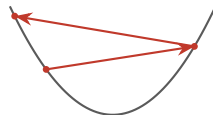
η too small



η about right



η too large



What the loss curve does

Diagnosis

Fix

falls, then flattens

correct

nothing

still falling steeply at the last epoch

η too small, or too few epochs

raise η or add epochs

rises, oscillates, or becomes NaN

η too large

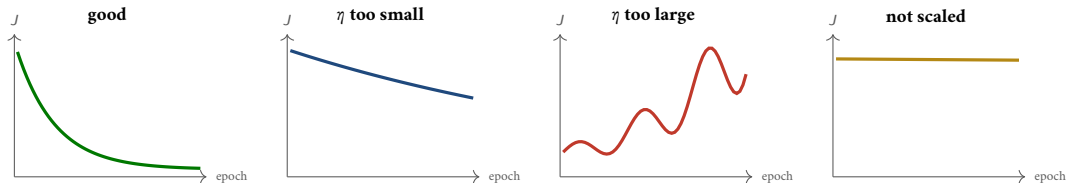
lower η

flat from the very first epoch

features not scaled

apply `StandardScaler`

The loss curve is the instrument panel



Habit to build

Plot the loss **every time** you train something. It costs one line and it is the fastest way to tell a broken run from a slow one. A model that trains without producing a loss curve is a model you cannot debug.

Terminology

- **one update** — the parameters θ move once. Also called a *step* or an *iteration*.
- **one epoch** — every row has been used once.
- **batch size** — how many rows are used to compute a single update.

$$\text{updates per epoch} = \frac{\text{rows}}{\text{batch size}}$$

The confusion to avoid

An epoch is not an update.

Batch GD: 1 update per epoch.

Mini-batch (32): 1,250 per epoch.

RMSE and R^2

RMSE — by how much are we typically wrong?

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

- Square the errors so signs cannot cancel, average, then square-root back to the original units.
- So it is measured **in CGPA points**: 0.51 is half a grade point.

R^2 — how much better than guessing the average?

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R^2	meaning
1.0	every prediction exactly right
0.74	the tutorial's model
0.0	no better than predicting $\bar{y}$
< 0	worse than predicting $\bar{y}$

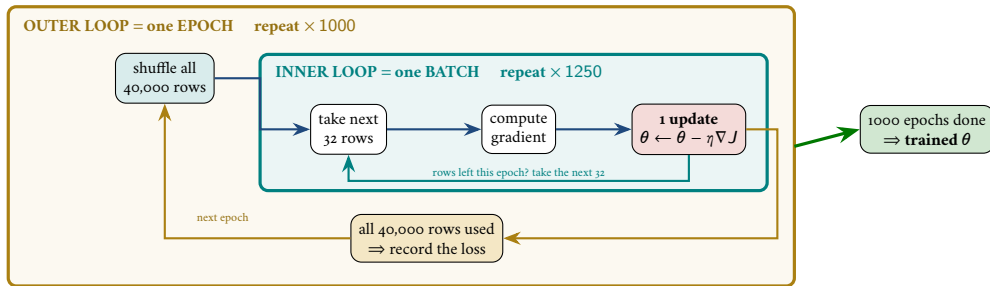
Key Insight

The denominator of R^2 is the error made by **always predicting the mean** — the regression baseline, just as “predict the commonest class” was the classification one. R^2 has no units and travels between problems; RMSE has units and is the one to quote to a person.

The whole training loop, in one picture

SETUP

done once



Read it as two nested loops: the **teal** box runs **1250 times** inside every pass of the **gold** box, which itself runs **1000 times**.

Side by side

	Batch	Mini-batch	Stochastic
Rows per update	all n	32 to 256	1
Updates per epoch	1	$n/32 = 1,250$	$n = 40,000$
Gradient quality	exact	good estimate	very noisy
Path downhill	smooth	slight wobble	very noisy
Memory needed	the whole set	one small batch	one row
Settles exactly?	yes	very close	bounces around
Used in practice?	small data only	yes, almost always	rarely alone

Key Insight

Batch and stochastic are the two extremes. Mini-batch is the practical compromise, and it is what the word “SGD” loosely means in every modern library.

MCQs

- A dataset has 10,000 rows and the batch size is 50. One epoch contains
- (a) 1 update (b) 50 updates (c) 200 updates (d) 10,000 updates

MCQs

- A dataset has 10,000 rows and the batch size is 50. One epoch contains
 - (a) 1 update (b) 50 updates (c) 200 updates (d) 10,000 updates
- Setting the batch size equal to n gives
 - (a) stochastic GD (b) batch GD (c) mini-batch GD (d) the normal equation

Answer: (c)

MCQs

- 1 A dataset has 10,000 rows and the batch size is 50. One epoch contains

(a) 1 update (b) 50 updates (c) 200 updates (d) 10,000 updates

Answer: (c)

- 2 Setting the batch size equal to n gives

(a) stochastic GD (b) batch GD (c) mini-batch GD (d) the normal equation

Answer: (b)

- 3 Your loss curve rises and then prints NaN. The most likely cause is

(a) too few epochs (b) the learning rate is too large (c) the batch size is too small (d) too much data

MCQs

- ❶ A dataset has 10,000 rows and the batch size is 50. One epoch contains

(a) 1 update (b) 50 updates (c) 200 updates (d) 10,000 updates

Answer: (c)

- ❷ Setting the batch size equal to n gives

(a) stochastic GD (b) batch GD (c) mini-batch GD (d) the normal equation

Answer: (b)

- ❸ Your loss curve rises and then prints NaN. The most likely cause is

(a) too few epochs (b) the learning rate is too large (c) the batch size is too small (d) too much data

Answer: (b)

- ❹ Gradient descent needs feature scaling but the normal equation does not, because

(a) the normal equation is newer (b) one learning rate must suit every direction at once (c) scaling changes the answer (d) matrices cannot be scaled

MCQs

- 1 A dataset has 10,000 rows and the batch size is 50. One epoch contains

(a) 1 update (b) 50 updates (c) 200 updates (d) 10,000 updates

Answer: (c)

- 2 Setting the batch size equal to n gives

(a) stochastic GD (b) batch GD (c) mini-batch GD (d) the normal equation

Answer: (b)

- 3 Your loss curve rises and then prints NaN. The most likely cause is

(a) too few epochs (b) the learning rate is too large (c) the batch size is too small (d) too much data

Answer: (b)

- 4 Gradient descent needs feature scaling but the normal equation does not, because

(a) the normal equation is newer (b) one learning rate must suit every direction at once (c) scaling changes the answer (d) matrices cannot be scaled

Answer: (b)